TP 5
Images, multimédia et mise en page CSS3 (Flexbox, Grid, tableaux)

Filière / Niveau	Licence Informatique — 1ère année
Module	Développement Web
Thème 1	Intégration Web Responsive
Durée	1 séance (3 h)
Prérequis	TP2 (tableaux HTML), TP4 (mise en forme CSS3), chapitre 4 du cours (images et objets multimédias), cycle Git add → commit → push

Objectifs du TP

À l'issue de ce TP, l'étudiant sera capable de :

- Insérer des **images** (`<img>`, `<figure>`, image cliquable avec `<map>` et `<area>`) et un **favicon**.
- Intégrer des objets **multimédias** : lecteur **audio** et lecteur **vidéo** avec plusieurs sources.
- Habiller une zone avec une **image de fond** (`background-image`, `background-repeat`, `background-position`…).
- Aligner des éléments sur une ligne ou sur une colonne avec **Flexbox**.
- Construire une grille de cartes avec **CSS Grid** : colonnes, fusion de cellules, zones nommées.
- Mettre en forme un **tableau** HTML : bordures fusionnées, en-têtes, lignes alternées, légende.
- **Relire et expliquer** la partie disposition de la feuille `mise-en-page.css` fournie au TP4.

I. Aperçu théorique

1.1 Les images en HTML

Rappel du TP2 : la balise `<img>` (sans balise fermante) s'écrit avec `src`, le chemin du fichier, et `alt`, le texte alternatif obligatoire, par exemple `<img src="images/logo.png" alt="Logo du club">`. Ce TP ajoute trois attributs :

Attribut	Rôle	Exemple
`width`, `height`	largeur, hauteur affichées (en pixels). Si une seule est donnée, l'autre est calculée en proportion ; sans les deux, l'image garde sa taille réelle.	`width="300"`
`title`	info-bulle affichée au survol	`title="Retour en haut"`

Formats courants : **JPG** pour les photographies, **PNG** pour les logos et les images avec transparence, **GIF** pour les petites animations.

a) Image-lien, figure et favicon

Une image placée **dans un lien** `<a>` devient cliquable. La balise `<figure>` regroupe une image et sa légende `<figcaption>`.

```
<a href="page1.html">
  <img src="image1.jpg" alt="Exemple de photo" title="Aller à la page 1">
</a>

<figure>
  <img src="images/trulli.jpg" alt="Trulli" width="300">
  <figcaption>Fig. 1 – Trulli, Pouilles, Italie.</figcaption>
</figure>
```

Le **favicon** est la petite image affichée dans l'onglet du navigateur. Il se déclare dans le `<head>` : `<link rel="icon" type="image/png" href="images/favicon.png">`. Le type dépend du format : `image/png`, `image/gif`, `image/x-icon` (fichier .ico).

b) L'image cliquable : <map> et <area>

Une **image cliquable** contient plusieurs zones, chacune menant à un lien. L'attribut `usemap="#nom"` de l'image la relie à la carte `<map name="nom">` ; chaque zone est une balise `<area>` qui précise sa forme (`shape`), ses coordonnées (`coords`), son lien (`href`) et son texte alternatif (`alt`).

shape	coords	Signification
`rect`	`x1,y1,x2,y2`	coins haut-gauche et bas-droit du rectangle
`circle`	`x,y,rayon`	centre et rayon du cercle
`poly`	`x1,y1,x2,y2,…,xn,yn`	sommets du polygone (fermé automatiquement)

```
<img src="workplace.jpg" alt="Bureau" usemap="#workmap" width="400" height="379">
<map name="workmap">
  <area shape="rect"   coords="34,44,270,350" href="computer.html" alt="Ordinateur">
  <area shape="circle" coords="337,300,44"    href="coffee.html"   alt="Tasse de café">
</map>
```

NB : les coordonnées sont en pixels, mesurées depuis le **coin haut-gauche** de l'image. L'image doit donc être affichée à sa taille réelle, sinon les zones ne correspondent plus au dessin.

1.2 L'audio et la vidéo

Les balises `<audio>` et `<video>` (HTML5) contiennent une ou plusieurs balises `<source>` : le navigateur lit **la première qu'il sait décoder**. Le texte placé juste avant la balise fermante n'est affiché que par les navigateurs qui ne connaissent pas la balise (texte de secours).

Attribut	Balises	Rôle
`controls`	`<audio>`, `<video>`	affiche les boutons : lecture / pause, avancement, volume
`loop`	`<audio>`, `<video>`	relance la lecture à la fin
`poster`	`<video>`	image affichée avant la lecture
`width`, `height`	`<video>`	dimensions du lecteur
`src`, `type`	`<source>`	fichier et type du média : `audio/mpeg` (MP3), `audio/ogg`, `audio/wav`, `video/mp4`, `video/webm`, `video/ogg`

Formats audio : **MP3**, **WAV**, **OGG**. Formats vidéo : **MP4**, **WebM**, **Ogg**. Autrefois, les navigateurs ne lisaient pas tous les mêmes formats. Aujourd'hui, MP4 et MP3 sont lus partout, mais on propose encore souvent plusieurs `<source>` pour offrir un format plus léger (WebM, Ogg) suivi d'un format de secours, du format préféré au moins préféré, en précisant toujours `type` : le navigateur saute alors immédiatement les fichiers qu'il ne sait pas lire. Deux autres attributs existent : `autoplay` (lecture automatique) et `muted` (son coupé au départ).

```
<video controls width="400" height="300">
  <source src="video1.webm" type="video/webm">
  <source src="video1.mp4"  type="video/mp4">
  <p>Votre navigateur ne lit pas les vidéos.
     <a href="video1.mp4">Lien vers la vidéo</a>.</p>
</video>

<audio controls>
  <source src="horse.ogg" type="audio/ogg">
  <source src="horse.mp3" type="audio/mpeg">
  Votre navigateur ne lit pas les fichiers audio.
</audio>
```

1.3 Les images de fond en CSS

Propriété	À quoi sert-elle ?	Valeurs courantes	Exemple
`background-image`	image de fond d'un élément	`url(…)`, `none`	`background-image: url("../images/fond.png");`
`background-repeat`	répétition de l'image	`repeat` (dans les deux sens), `repeat-x`, `repeat-y`, `no-repeat`	`background-repeat: no-repeat;`
`background-position`	position de l'image	mots-clés (`left`, `center`, `right`, `top`, `bottom`), `%`, `px`	`background-position: right top;`
`background-attachment`	l'image défile avec la page ou reste fixe	`scroll`, `fixed`	`background-attachment: fixed;`
`background-size`	taille de l'image	largeur puis hauteur, en `px` ou `%`	`background-size: 150px 80px;`
`background`	raccourci : tout en une ligne	couleur, image, répétition, position	`background: #fff url("tree.jpg") no-repeat right top;`

Avec deux valeurs de position, la **première est horizontale** : `0% 0%` place l'image dans le coin haut-gauche, `100% 100%` dans le coin bas-droit, `50% 50%` au centre. Prévoir toujours une **couleur de fond** (`background-color`) : elle reste visible si l'image ne se charge pas.

NB : comme au TP4, le chemin de l'image est relatif **au fichier CSS** : depuis `css/mise-en-page.css`, on écrit `url("../images/motif.png")`.

1.4 Le type d'affichage : bloc, en ligne… flex et grid

Un élément de type **bloc** (`<div>`, `<p>`, `<h1>`, `<section>`…) occupe toute la largeur et passe à la ligne ; un élément **en ligne** (`<span>`, `<a>`, `<img>`…) se place dans le fil du texte. La propriété `display` change ce comportement (`display: block;` ou `display: inline;`). Deux autres valeurs servent à **disposer** les blocs : `display: flex;` (partie 1.5) et `display: grid;` (partie 1.6).

1.5 Flexbox : aligner sur une ligne ou une colonne

On écrit `display: flex;` sur le **conteneur** (le parent) : ses **enfants directs** deviennent des éléments flexibles et se placent côte à côte sur l'**axe principal** (horizontal par défaut). L'axe perpendiculaire est l'**axe secondaire**.

```
.menu {              /* le conteneur : la liste <ul>         */
  display: flex;     /* ses <li> se placent sur une ligne     */
  gap: 10px;         /* 10px d'espace entre deux éléments     */
}
```

a) Les propriétés du conteneur

Propriété	À quoi sert-elle ?	Valeurs courantes
`display: flex`	active Flexbox sur le conteneur	—
`display: inline-flex`	comme `flex`, mais le conteneur reste **en ligne** : il ne prend que la largeur de son contenu	— (par exemple un bouton avec une icône)
`flex-direction`	sens de l'axe principal	`row` (ligne, par défaut), `column` (colonne)
`justify-content`	répartition sur l'axe principal	`flex-start`, `center`, `flex-end`, `space-between` (espace égal entre les éléments), `space-around`
`align-items`	alignement sur l'axe secondaire	`stretch` (étirés, par défaut), `flex-start`, `center`, `flex-end`
`flex-wrap`	passage à la ligne quand la place manque	`nowrap` (par défaut), `wrap`
`gap`	espace entre les éléments	`10px`, `24px`…

b) Les propriétés d'un élément

Propriété	À quoi sert-elle ?	Exemple
`flex-grow`	part de l'espace libre prise par l'élément (agrandissement)	`flex-grow: 1;` (prend toute la place restante)
`flex-shrink`	part de rétrécissement quand la place manque (`0` : ne rétrécit pas)	`flex-shrink: 0;`
`flex-basis`	taille de départ de l'élément, avant le partage de l'espace libre	`flex-basis: 40%;`
`flex`	raccourci, dans l'ordre : agrandissement, rétrécissement, taille de départ	`flex: 1 1 40%;` ; `flex: 1;` (= `flex-grow: 1`) ; `flex: none;` (ni agrandi, ni rétréci)
`align-self`	alignement de **cet élément seul** sur l'axe secondaire	`align-self: flex-start;`
`order`	ordre d'affichage	`order: -1;` (passe en premier)

1.6 Grid : construire une grille

`display: grid;` découpe le conteneur en **colonnes** et en **lignes**. L'unité `fr` (fraction) partage l'espace disponible : `1fr 2fr` donne une deuxième colonne deux fois plus large que la première ;

`repeat(3, 1fr)` donne trois colonnes égales. Un enfant peut occuper plusieurs cases (`span`) ou une **zone nommée**.

Propriété	À quoi sert-elle ?	Exemple
`display: grid`	active la grille sur le conteneur	—
`grid-template-columns`	nombre et largeur des colonnes	`200px 1fr`, `repeat(3, 1fr)`
`grid-template-rows`	hauteur des lignes	`100px auto`
`gap`	espace entre les cases	`gap: 20px;`
`grid-column`, `grid-row`	nombre de colonnes / de lignes occupées par un enfant	`grid-column: span 2;`
`grid-template-areas`	dessin de la grille avec des zones nommées (une chaîne par ligne)	`"haut haut" "gauche droite"`
`grid-area`	place un enfant dans une zone nommée	`grid-area: haut;`

```
.page {
  display: grid;
  grid-template-columns: 1fr 3fr;        /* 2 colonnes : 1/4 et 3/4 */
  grid-template-areas:
    "entete  entete"                     /* 1re ligne : sur les deux colonnes */
    "menu    contenu";                   /* 2e ligne : une zone par colonne    */
  gap: 20px;
}
header { grid-area: entete; }
nav    { grid-area: menu; }
main   { grid-area: contenu; }
```

1.7 Mettre en forme un tableau

Rappel du TP2 : conteneur `<table>`, titre `<caption>`, groupes `<thead>` (ligne des titres de colonnes) et `<tbody>` (lignes de données), lignes `<tr>`, cellules d'en-tête `<th>` et de données `<td>`.

Nouveau dans ce TP : le groupe `<tfoot>` contient la ou les lignes de **pied de tableau** (une ligne de total, par exemple). L'attribut `colspan="n"` fusionne une cellule sur **n colonnes** ; `rowspan="n"` la fusionne sur **n lignes**.

```
<table>
  <caption>Inscriptions</caption>
  <thead>
    <tr><th>Atelier</th><th>Jour</th><th>Places</th></tr>
  </thead>
  <tbody>
    <tr><td>Python</td><td rowspan="2">Lundi</td><td>20</td></tr>
    <tr><td>Web</td><td>18</td></tr>
  </tbody>
  <tfoot>
    <tr><td colspan="2">Total</td><td>38</td></tr>
  </tfoot>
</table>
```

Ici, la cellule « Lundi » occupe deux lignes (`rowspan="2"`) : la deuxième ligne n'a donc que deux cellules. La cellule « Total » occupe deux colonnes (`colspan="2"`). Le CSS met ensuite le tableau en forme :

Propriété ou sélecteur	À quoi sert-il ?	Exemple
`border-collapse`	fusionne les bordures de deux cellules voisines en un seul trait	`border-collapse: collapse;`
`border-spacing`	espace entre les cellules (bordures séparées)	`border-spacing: 4px;`
`caption-side`	position de la légende	`top`, `bottom`
`padding` sur `th, td`	marge intérieure des cellules	`padding: 10px 14px;`
`text-align`, `vertical-align`	alignement horizontal / vertical du contenu	`text-align: center;`
`tr:nth-child(even)`	cible les lignes paires (`odd` : impaires)	`tbody tr:nth-child(even) { … }`
`tr:hover`	ligne survolée par la souris	`tbody tr:hover { … }`
`td:last-child`	dernière cellule de chaque ligne	`td:last-child { text-align: center; }`

À retenir

`display: flex` et `display: grid` s'écrivent sur le **parent** ; `flex-grow`, `align-self`, `grid-column` et `grid-area` s'écrivent sur un **enfant**.

Flexbox range des éléments dans **une seule direction** (une ligne ou une colonne) : menu, barre d'outils, contenu d'une carte. **Grid** organise **deux directions à la fois** (lignes et colonnes) : galerie, grille de cartes, gabarit de page.

Les outils de développement (**F12**, onglet *Éléments*) affichent un badge `flex` ou `grid` à côté du conteneur : un clic dessine les axes et les cases de la grille.

II. Application pratique

Dans ce TP, le dossier et le dépôt s'appellent **TP5**. Un seul exercice : compléter et mettre en page la page `ateliers.html` du Club Informatique, dont l'aperçu est donné plus bas. Le HTML est **fourni** avec ses classes, **sauf les images et les médias**, que vous insérez (partie A). Le fichier `base.css` reprend les polices, couleurs et marges du TP4 : **ne le modifiez pas**. **Vous écrivez** `mise-en-page.css` : images de fond, Flexbox, Grid et tableau (partie B).

Consignes

a. *Les images et les médias doivent être insérés dans* `ateliers.html` *aux emplacements indiqués par les commentaires* `<!-- Étape … -->`.

b. *Toutes les règles CSS doivent être enregistrées dans le fichier* `mise-en-page.css` *du dossier* `css`.

c. *Le fichier* `base.css` *est fourni : il ne doit pas être modifié.*

d. *Tous les chemins doivent être relatifs (*`images/…`*,* `medias/…`*) : jamais de chemin du type* `C:\…`.

e. *Chaque modification doit être testée : enregistrer (Ctrl + S), puis actualiser la page dans le navigateur (F5).*

2.1 Préparation et Git (10 min)

- Décompresser `TP5_fichiers_etudiants.zip` dans votre dossier `TPWeb` : le dossier **TP5** est créé avec la bonne organisation. **Ne déplacez aucun fichier.**
- Créer le fichier vide `mise-en-page.css` dans le dossier `css`. Vous devez obtenir exactement cette structure :

```
TP5/
├── ateliers.html          ← à la racine de TP5
├── css/
│   ├── base.css           ← fourni, ne pas modifier
│   └── mise-en-page.css   ← votre fichier (vide au départ)
├── images/
│   ├── logo.svg   favicon.png   motif.png   banniere.png   plan-salles.png
│   └── ateliers/  (python.jpg, web.jpg, jeux.jpg, robotique.jpg, cyber.jpg, ia.jpg)
└── medias/
    └── presentation.mp4   poster.jpg   podcast.mp3   podcast.wav
```

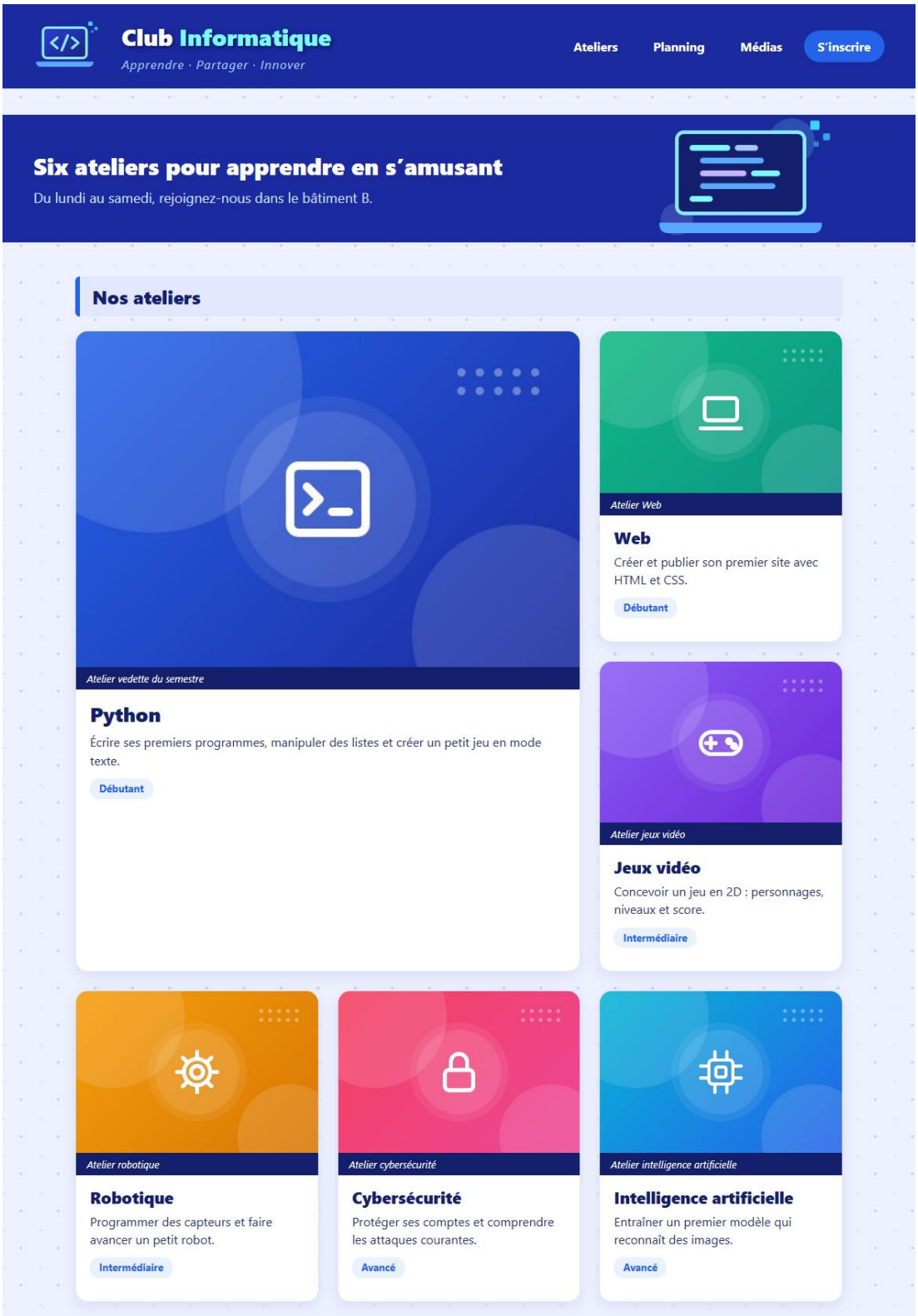
- Ouvrir un terminal dans le dossier `TP5`, initialiser le dépôt Git, créer sur GitHub un dépôt **vide** nommé `TP5` et le relier :

```
cd TP5
git init
git branch -M main
git remote add origin <URL_du_dépôt>
```

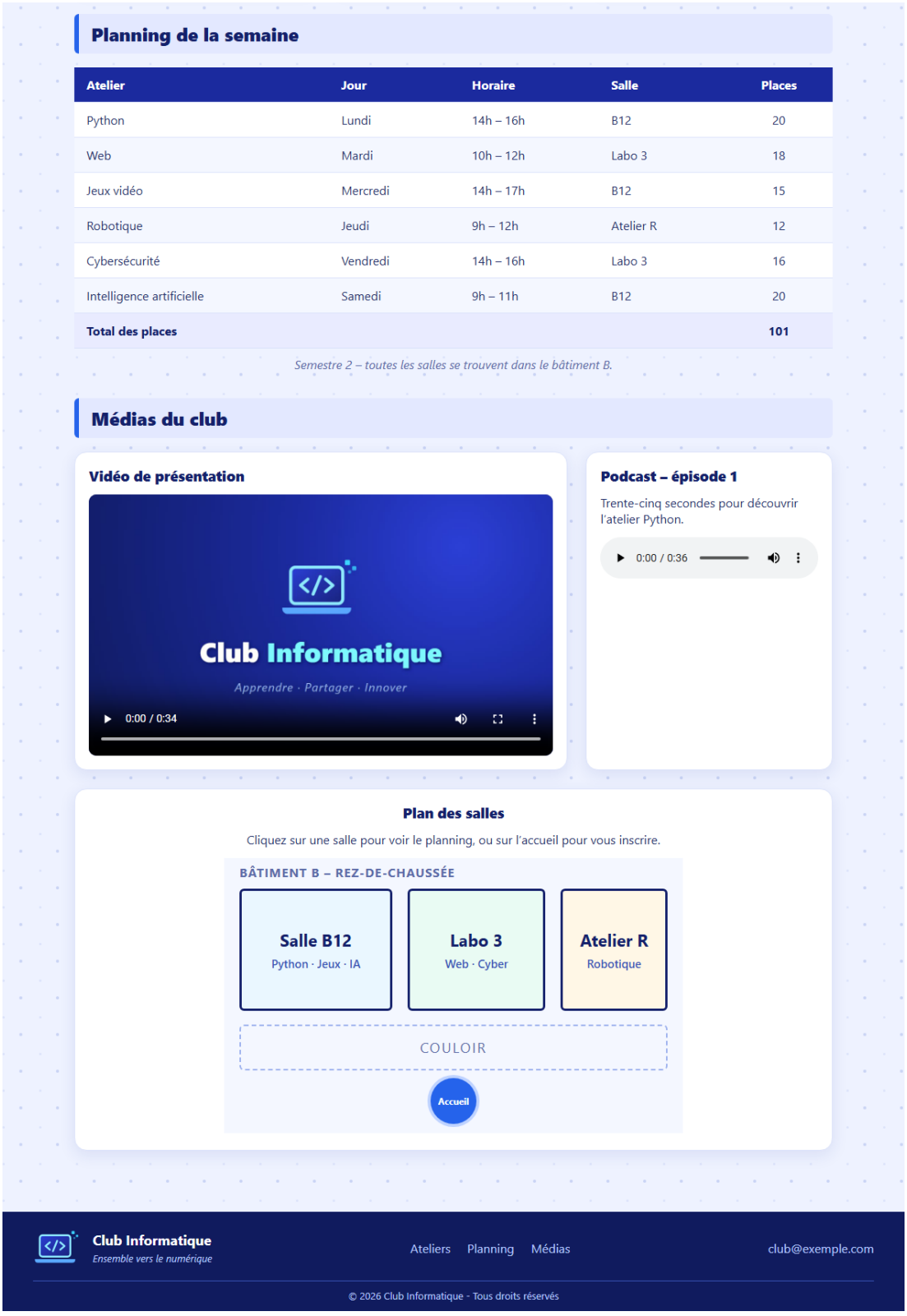
- Ouvrir `ateliers.html` dans le navigateur : les textes sont déjà mis en forme, mais les blocs sont empilés, la bannière est illisible et les images manquent. La page se transformera à chaque étape (enregistrer, puis **F5**).

2.2 Exercice : Page « Ateliers » du Club Informatique

Réalisez la page dont l'aperçu est donné ci-dessous.



Aperçu (1/2) : en-tête, bannière et grille des ateliers



Aperçu (2/2) : planning, médias et pied de page

Observez l'aperçu et repérez les zones : en-tête (logo, titre, menu), bannière, grille de six cartes dont une grande carte « vedette », tableau du planning, section médias (vidéo, podcast, plan des salles), pied de page. Ouvrez ensuite `ateliers.html` et repérez les classes (`entete`, `menu`,

`banniere`, `grille-ateliers`, `atelier`, `vedette`, `infos`, `niveau`, `planning`, `grille-medias`, `pied-haut`…).

> **Comment lire les tableaux**
> Comme au TP4, chaque zone est décrite par un tableau à trois colonnes.
>
> **Partie A (HTML)** : *Élément* | *Attribut* | *Valeur*. L'attribut est décrit par son **rôle** (« Texte alternatif », « Afficher les boutons de lecture ») : retrouvez son nom dans la partie théorique (1.1 et 1.2), puis écrivez la balise complète.
>
> **Partie B (CSS)** : *Élément* | *Propriété* | *Valeur*. Écrivez le **sélecteur** à partir de la description de l'élément, puis la propriété qui produit l'effet attendu (parties 1.3 à 1.7). Quand une notion n'a pas été vue en cours, son nom est donné entre parenthèses.
>
> **Exemples** : la ligne « Logo (`<img>`) | Largeur | `90` » donne l'attribut `width="90"` ; la ligne « Menu (classe `menu`) | Espace entre les éléments | `10px` » donne la déclaration `gap: 10px;` dans la règle `.menu`.

Partie A – HTML : les images et les objets multimédias (25 min)

Étape 1 : les images (15 min)

a. Dans le `<head>`, remplacer le commentaire `<!-- Étape 1.a -->` par le **favicon** du club :

Élément	Attribut (rôle)	Valeur
Lien vers l'icône (balise `<link>`)	Relation avec la page	`icon`
	Type de l'image	`image/png`
	Chemin du fichier	`images/favicon.png`

b. Dans l'en-tête, remplacer le commentaire `<!-- Étape 1.b -->` par le **logo cliquable** : un lien qui ramène en haut de la page et qui contient l'image du logo.

Élément	Attribut (rôle)	Valeur
Lien (balise `<a>`)	Destination du lien	`#haut` (le haut de la page)
	Info-bulle	Revenir en haut de la page
Logo (balise `<img>`, classe `logo`) *Placé à l'intérieur du lien.*	Chemin du fichier	`images/logo.svg`
	Texte alternatif	Logo du Club Informatique
	Largeur	`90`

c. Dans la carte vedette (Python), la balise `<figure>` contient déjà sa légende `<figcaption>`. Remplacer le commentaire `<!-- Étape 1.c -->` (avant la légende) par l'image de l'atelier. Les cinq autres cartes contiennent déjà la leur : observez-les.

Élément	Attribut (rôle)	Valeur
Image d'un atelier (balise `<img>`)	Chemin du fichier	`images/ateliers/python.jpg`
	Texte alternatif	Illustration de l'atelier Python
	Largeur	`300`
	Hauteur	`200`

Remarque : `base.css` adapte ensuite la largeur affichée à celle de la carte. Les attributs `width` et `height` restent utiles : le navigateur réserve la place de l'image avant même de l'avoir chargée.

d. Dans la section Médias, remplacer le commentaire `<!-- Étape 1.d -->` par le **plan cliquable des salles** :

- ✓ l'image `plan-salles.png` (600 × 360 pixels), reliée à une carte nommée `salles` ;
- ✓ la carte `<map name="salles">` est déjà écrite et contient les zones « Labo 3 » et « Atelier R » : y **ajouter les deux zones manquantes**, le rectangle de la salle B12 (vers le planning) et le cercle de l'accueil (vers la page d'inscription du TP4).

Élément	Attribut (rôle)	Valeur
Image du plan (balise `<img>`)	Chemin du fichier	`images/plan-salles.png`
	Texte alternatif	Plan des salles du bâtiment B
	Largeur et hauteur	`600` et `360`
	Carte associée	`#salles`
Zone « Salle B12 » (balise `<area>`)	Forme	Rectangle
	Coordonnées	`20,40,220,200`
	Destination du lien	`#planning`
	Texte alternatif	Salle B12
Zone « Accueil » (balise `<area>`)	Forme	Cercle
	Coordonnées	centre `300,318`, rayon `30`
	Destination du lien	`../TP4/inscription.html` (la page d'inscription du TP4, dans le dossier voisin)
	Texte alternatif	Accueil : s'inscrire au club

e. Cliquer sur chaque salle, puis sur l'accueil, pour vérifier les liens. Que se passe-t-il si l'on écrit la carte associée sans le caractère `#` ?

Étape 2 : l'audio et la vidéo (10 min)

a. Dans la carte « Vidéo de présentation », remplacer le commentaire `<!-- Étape 2.a -->` par un **lecteur vidéo** :

- ✓ un lecteur `<video>` avec ses boutons, de taille 640 × 360, qui affiche l'image `poster.jpg` avant la lecture ;
- ✓ une source `<source>` : le fichier `presentation.mp4` ;
- ✓ un paragraphe de secours : « Votre navigateur ne lit pas les vidéos HTML5. »

Élément	Attribut (rôle)	Valeur
Lecteur vidéo (balise `<video>`)	Afficher les boutons de lecture	(attribut sans valeur)
	Largeur	`640`
	Hauteur	`360`
	Image affichée avant la lecture (`poster`)	`medias/poster.jpg`
Source (balise `<source>`)	Chemin du fichier	`medias/presentation.mp4`
	Type du média	`video/mp4`

b. Dans la carte « Podcast », remplacer le commentaire `<!-- Étape 2.b -->` par un **lecteur audio** qui propose deux formats, suivis du texte de secours « Votre navigateur ne lit pas les fichiers audio. » :

Élément	Attribut (rôle)	Valeur
Lecteur audio (balise `<audio>`)	Afficher les boutons de lecture	(attribut sans valeur)
Première source (balise `<source>`)	Chemin du fichier	`medias/podcast.mp3`
	Type du média	`audio/mpeg`
Deuxième source (balise `<source>`)	Chemin du fichier	`medias/podcast.wav`
	Type du média	`audio/wav`

c. Pourquoi propose-t-on deux sources ? Dans quel ordre le navigateur les essaie-t-il, et à quoi sert l'attribut « Type du média » ? Ajouter ensuite au lecteur audio l'attribut qui **relance la lecture à la fin**, tester, puis le retirer.

Partie B – CSS : la mise en page (2 h 10)

Étape 3 : les images de fond (20 min)

a. Ajouter au fichier `mise-en-page.css`, les règles permettant d'appliquer à la page et à la bannière les fonds spécifiés dans le tableau suivant :

Élément	Propriété	Valeur
Corps de la page (balise `<body>`)	Couleur d'arrière-plan	`#EEF2FF`
	Image de fond	`../images/motif.png`
	Répétition de l'image	Dans les deux sens
	Défilement de l'image	Fixe (l'image ne défile pas avec la page)
Bannière (classe `banniere`)	Couleur d'arrière-plan (couleur de secours)	`#1B2A9E`
	Image de fond	`../images/banniere.png`
	Répétition de l'image	Aucune
	Position de l'image	`90%` horizontalement, `50%` verticalement
	Taille de l'image	`270px` de large, `180px` de haut

b. Pourquoi le chemin des images commence-t-il par `../` ? Faire défiler la page : que fait le motif du fond ? Remplacer « Fixe » par la valeur inverse et comparer.

Étape 4 : Flexbox (30 min)

a. Observer, dans `ateliers.html`, l'organisation des zones à aligner :

- ✓ L'en-tête `<header class="entete">` contient trois enfants : le lien du logo, un bloc `<div class="titre-bloc">` (titre `<h1>` et devise) et la navigation `<nav>`.
- ✓ Le menu est une liste `<ul class="menu">` de quatre éléments `<li>`, chacun contenant un lien.
- ✓ Chaque carte contient un bloc `<div class="infos">` : un titre `<h3>`, une description `<p>` et un badge de niveau `<p class="niveau">`.
- ✓ Dans le pied de page, le bloc `<div class="pied-haut">` contient trois enfants : la marque `<div class="marque">` (logo et nom du club), la liste de liens `<ul class="liens-pied">` et le contact `<p class="contact">`.

b. Ajouter au fichier `mise-en-page.css`, les règles permettant d'appliquer à ces éléments les dispositions spécifiées dans le tableau suivant :

Élément	Propriété	Valeur
En-tête (classe `entete`)	Type d'affichage	Flexbox
	Alignement sur l'axe secondaire	Centré
	Espace entre les éléments	`24px`
Bloc du titre (classe `titre-bloc`)	Part de l'espace libre	`1` (prend toute la place restante)
Menu (classe `menu`)	Type d'affichage	Flexbox
	Espace entre les éléments	`10px`
Bloc d'informations d'une carte (classe `infos`)	Type d'affichage	Flexbox
	Sens de l'axe principal	Colonne
Badge de niveau (classe `niveau`)	Alignement de cet élément seul sur l'axe secondaire	Au début
Partie haute du pied (classe `pied-haut`)	Type d'affichage	Flexbox
	Répartition sur l'axe principal	Espace égal entre les éléments
	Alignement sur l'axe secondaire	Centré
Marque (classe `marque`)	Type d'affichage	Flexbox
	Alignement sur l'axe secondaire	Centré
	Espace entre les éléments	`14px`
Liste des liens du pied (classe `liens-pied`)	Type d'affichage	Flexbox
	Espace entre les éléments	`22px`

c. Avant la règle du badge, celui-ci occupait toute la largeur de la carte : pourquoi ? (aide : valeur par défaut de l'alignement sur l'axe secondaire).

d. Dans la règle de l'en-tête, remplacer « Centré » par « Au début », observer, puis rétablir.

Étape 5 : Grid (30 min)

a. Observer l'organisation des deux grilles :

- ✓ Les six cartes `<article class="atelier">` sont regroupées dans `<div class="grille-ateliers">` ; la première porte aussi la classe `vedette`.
- ✓ Les trois cartes de la section Médias (`<article class="media video">`, `media audio` et `media plan`) sont regroupées dans `<div class="grille-medias">`.

b. Ajouter au fichier `mise-en-page.css`, les règles permettant de construire les grilles spécifiées dans le tableau suivant :

Élément	Propriété	Valeur
Grille des ateliers (classe `grille-ateliers`)	Type d'affichage	Grille
	Colonnes	`3` colonnes de même largeur
	Espace entre les cases	`24px`
Carte vedette (classe `vedette`)	Nombre de colonnes occupées	`2`
	Nombre de lignes occupées	`2`
Grille des médias (classe `grille-medias`)	Type d'affichage	Grille
	Colonnes	`2` colonnes, la première deux fois plus large que la seconde
	Zones nommées	1re ligne : `video` puis `audio` ; 2e ligne : `plan` sur les deux colonnes
	Espace entre les cases	`24px`
Cartes des médias (classes `video`, `audio` et `plan`) *Une règle par carte.*	Zone occupée	la zone du même nom

c. Combien de cases compte la grille des ateliers ? Vérifier que les six cartes, dont une de 2 × 2 cases, la remplissent exactement.

d. Pourquoi le plan des salles occupe-t-il toute une ligne de la grille des médias ? (aide : partie 1.1, NB sur les coordonnées des zones).

Étape 6 : le tableau du planning (30 min)

a. Observer le tableau `<table class="planning">` : une légende `<caption>`, une ligne d'en-têtes `<th>` dans `<thead>`, six lignes dans `<tbody>` et une ligne de total dans `<tfoot>`, dont la première cellule fusionne quatre colonnes (`colspan="4"`).

b. Ajouter au fichier `mise-en-page.css`, les règles permettant d'appliquer au tableau les mises en forme spécifiées dans le tableau suivant :

Élément	Propriété	Valeur
Tableau (classe `planning`)	Largeur	`100%`
	Bordures de deux cellules voisines	Fusionnées en un seul trait
	Couleur d'arrière-plan	Blanc

Élément	Propriété	Valeur
Légende du tableau (`<caption>`)	Position de la légende	Sous le tableau
	Marge intérieure en haut seulement	`10px`
	Style de police	Italique
	Couleur du texte	`#5B6BA8`
Cellules d'en-tête et de données (`<th>` et `<td>`) *Une seule règle.*	Marge intérieure	`12px` en haut et en bas, `16px` à gauche et à droite
	Bordure en bas seulement	`1px`, trait plein, `#DCE3FB`
	Alignement du texte	À gauche
Cellules d'en-tête (`<th>`)	Couleur du texte	Blanc
	Couleur d'arrière-plan	`#1B2A9E`
Lignes paires du corps du tableau	Couleur d'arrière-plan	`#F4F7FF`
Lignes du corps du tableau, au survol	Couleur d'arrière-plan	`#E3E8FF`
Dernière cellule de chaque ligne, en-têtes compris	Alignement du texte	Centré
Cellules du pied du tableau (`<td>` dans `<tfoot>`)	Graisse	Gras
	Couleur du texte	`#14206B` *(marine)*
	Couleur d'arrière-plan	`#E9EBFF`

c. Supprimer provisoirement la déclaration « Bordures fusionnées » : que constatez-vous entre les cellules ? La rétablir.

d. Ajouter une ligne au milieu du `<tbody>` : les couleurs des lignes paires sont-elles recalculées ? Pourquoi ? Supprimer ensuite cette ligne.

Étape 7 : relire la mise en page du TP4 (20 min)

Au TP4, le fichier `mise-en-page.css` vous était fourni avec la mention « notions vues dans les prochains chapitres ». Vous pouvez maintenant en lire la partie **disposition**. Ouvrir `TP4/css/mise-en-page.css` **sans le modifier**, et retrouver les règles suivantes (extraits) :

```
.entete {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1.8rem;
}
.titre-bloc { flex: 1; }

.grille {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem 1.6rem;
}
.champ { flex: 1 1 40%; }

.duo        { display: flex; gap: 1rem; }
.duo .carte { flex: 1; }
.duo .rose  { flex: 1.25; }

button { display: inline-flex; align-items: center; }
```

> **Aide à la lecture**
>
> `rem` est une unité relative : `1rem` vaut la taille de police de la page (16px par défaut), donc `1.8rem` ≈ 29px.
>
> `main { max-width: 920px; margin: 1.5rem auto; }` limite la largeur du contenu et le **centre** : la marge `auto` partage l'espace restant entre la gauche et la droite.
>
> `.carte > label` : le signe `>` cible les `<label>` **enfants directs** de `.carte` (pas ceux placés plus profondément).

a. **En-tête** : quels sont les trois enfants du conteneur `.entete` dans `inscription.html` ? Quelle déclaration les centre verticalement ? Pourquoi l'accroche se retrouve-t-elle collée à droite ? (aide : la règle `.titre-bloc`).

b. **Les deux colonnes de champs** : `.grille` contient six paragraphes `.champ`. À l'aide de `flex-wrap` et de `flex: 1 1 40%`, expliquer pourquoi les champs se placent **deux par ligne**. Combien y en aurait-il par ligne avec `flex: 1 1 25%` ?

c. **Le bloc duo** : quelle carte est la plus large, « Année d'études » (`violet`) ou « Motivation » (`rose`) ? Dans quelle proportion se partagent-elles l'espace libre ?

d. **Les boutons** : quelle différence y a-t-il entre `display: inline-flex` et `display: flex` ? Que range `align-items: center` à l'intérieur d'un bouton ?

e. **Bilan** : relever dans le fichier trois autres conteneurs flex (par exemple `.options`, `.actions`, `.pied-haut`) et indiquer, pour chacun, dans quelle direction ses enfants sont rangés. Les règles restantes (`::before`, `linear-gradient`, `box-shadow`, `transition`…) relèvent des prochains chapitres.

> **Vérification et commit (10 min)**
> Comparer votre page à l'aperçu zone par zone. Tester : le favicon dans l'onglet, le clic sur le logo, chaque zone du plan des salles, la lecture de la vidéo et du podcast, le survol des lignes du planning et des liens du menu.

Cycle Git : `git add .` → `git commit -m "Images, médias et mise en page CSS de la page Ateliers"` → `git push -u origin main`.

2.3 Pour aller plus loin

- **Flexbox** : réduisez la largeur de la fenêtre : les éléments de l'en-tête se serrent. Ajoutez à `.menu` le passage à la ligne (`flex-wrap`) et observez. Avec `order`, faites passer le bouton « S'inscrire » en premier.
- **Grid** : remplacez les 3 colonnes de la grille des ateliers par 4 colonnes : comment se placent les cartes ? Que faut-il changer pour remplir de nouveau la grille sans case vide ?
- **Fonds** : donnez à la bannière un **deuxième fond** (fonds multiples, chapitre 4) : le motif `motif.png` répété, placé sous l'illustration.
- **Image cliquable** : ajoutez au plan une zone **polygonale** (`shape="poly"`) sur le couloir (de `20,218` à `580,278`), avec un lien vers `#medias`.